

Elixir Alongside Erlang — A Readable Companion to the OTP Docs

Chapter 12 · checked against OTP 29.1.1 and the Elixir docs · Sep 25, 2026

1. Why the previous eleven chapters apply to Elixir unchanged

Elixir compiles to the same BEAM bytecode and runs on the same runtime as Erlang. Processes, mailboxes, binaries, ETS, supervision, distribution and schedulers are not Elixir features with Erlang equivalents; they are the same features. Every runtime behaviour in chapters 1–11 holds exactly in Elixir.

What Elixir adds is a layer of convenience: macros that generate child specs, abstractions like `Task` and `Agent`, and a build tool that handles configuration and releases. Convenience hides defaults, and the defaults are where plausible-but-wrong code comes from. This chapter maps the two languages and points out where Elixir’s defaults bite.

2. The building blocks

The map between the two

Concept	Erlang	Elixir
Generic server	<code>gen_server</code>	<code>GenServer</code> (use <code>GenServer</code>)
State machine	<code>gen_statem</code>	<code>:gen_statem</code> directly (no core wrapper)
Supervisor	<code>supervisor</code>	<code>Supervisor</code>
Dynamic children	<code>simple_one_for_one</code>	<code>DynamicSupervisor</code>
One-off work with a result	<code>spawn_monitor</code> + <code>receive</code>	<code>Task.async</code> / <code>Task.await</code>
Supervised one-off work	a <code>simple_one_for_one</code> of workers	<code>Task.Supervisor</code>
State held in a process	a small <code>gen_server</code>	<code>Agent</code>
Process registry	<code>global</code> , <code>pg</code> , or libraries	<code>Registry</code> (local, built on ETS)
Application	<code>.app.src</code> + application callback	<code>mix.exs</code> + <code>Application</code>
Configuration	<code>sys.config</code>	<code>config/*.exs</code> , <code>config/runtime.exs</code>
Releases	<code>rebar3</code> / <code>relx</code>	<code>mix release</code>
Unit tests	<code>EUnit</code> , <code>Common Test</code>	<code>ExUnit</code>

Concept	Erlang	Elixir
Strings	lists or binaries	binaries (<code>"..."</code>); charlists are <code>~c"..."</code>
Calling the other language	<code>'Elixir.Module':fun()</code>	<code>:module.fun()</code>

What `use GenServer` generates

`use GenServer` defines `child_spec/1` for you, with `restart: :permanent` and `shutdown: 5000` by default. Override them in the `use` line:

```
use GenServer, restart: :temporary, shutdown: 10_000
```

`Supervisor` and `DynamicSupervisor` read this spec, so the defaults decide what happens on crash, exactly as in chapter 1.

A guiding rule from the Elixir docs

“Use processes only to model runtime properties, such as mutable state, concurrency and failures, never for code organization.” A module of pure functions is not improved by wrapping it in a `GenServer`; it only gains a bottleneck.

3. How the runtime actually does it

Task.async is a linked, monitored child

`Task.async` starts a process that is both linked to and monitored by the caller. The docs spell out the consequence: if the caller crashes, the task crashes too, and vice versa. The reply is always sent, so the caller must `Task.await` it (default timeout 5000 ms). This is right when the result is part of the caller’s work, and wrong when a failure in the task should not take the caller down.

Agent functions run inside the agent

The function you pass to `Agent.get/2` or `Agent.update/2` runs in the agent process, not in the caller. The docs describe the trade-off: expensive work inside the agent blocks every other client; moving it to the client means the state may change in the meantime.

Configuration has two times

Anything in `config/config.exs` and friends is evaluated at build time and baked into the release; `config/runtime.exs` is evaluated when the release boots. Module attributes (`@x`) are evaluated at compile time (chapter 7, section 4.2).

4. Plausible but wrong

4.1 Task.async for work that may fail

```
def handle_call({:enrich, user}, _from, state) do
  task = Task.async(fn -> ExternalAPI.fetch_profile(user.id) end)
  {:reply, Task.await(task), state}
end
```

If the external call raises, the linked task takes the `GenServer` down with it, losing its state because of one bad response. Awaiting inside `handle_call` also blocks the server for up to 5 seconds (chapter 5, section 4.1). Fix: `Task.Supervisor.async_nolink/2` and handle `{:exit, reason}` explicitly, or do the call in the client.

4.2 An Agent doing the work

```
Agent.get(Stats, fn events -> compute_percentiles(events) end)
```

`compute_percentiles/1` runs inside the agent. Every other client waits behind it, and a slow computation turns the agent into the bottleneck of the system. Fix: read the data out (`Agent.get(Stats, &&1)`) and compute in the caller. If the data is read-mostly, put it in ETS instead (chapter 4).

4.3 A GenServer used as a namespace

```
defmodule Tax do
  use GenServer
  def calculate(amount), do: GenServer.call(__MODULE__, {:calc, amount})
  def handle_call({:calc, a}, _from, s), do: {:reply, a * 0.2, s}
end
```

There is no state, no concurrency and no failure to model, only a single process every caller now queues behind. This is a very common shape in AI-generated Elixir. Fix: `def calculate(amount), do: amount * 0.2`.

4.4 Per-connection workers left permanent

```
defmodule Conn do
  use GenServer          # restart: :permanent by default
end

DynamicSupervisor.start_child(ConnSup, {Conn, socket})
```

When a client disconnects abnormally, the supervisor restarts the worker with the same, now-closed socket; it crashes again, and repeated crashes can exhaust the supervisor's restart intensity (chapter 1, section 4.5). Fix: `use GenServer, restart: :temporary` for per-connection and per-request processes.

4.5 Atoms from input

```
def handle_params(%{"sort" => field}, _uri, socket) do
  {:noreply, assign(socket, :sort, String.to_atom(field))}
end
```

Every distinct value becomes a permanent atom; enough requests exhaust the atom table and the node goes down (chapter 3, section 4.5). Fix: `String.to_existing_atom/1`, or better, an explicit allow-list:

```
@sortable %{"name" => :name, "date" => :date}
sort = Map.fetch!(@sortable, field)
```

4.6 Loading everything to process some of it

```
File.read!("events.log")
|> String.split("\n")
|> Enum.filter(&String.contains?(&1, "ERROR"))
|> Enum.take(10)
```

For a 5 GB file this reads everything into memory, splits all of it, and filters all of it just to return ten lines. Fix: `File.stream!/1` with `Stream.filter/2` and `Enum.take/2`, which reads lazily and stops as soon as ten matches are found.

4.7 Runtime secrets in build-time config

```
# config/prod.exs
config :my_app, api_key: System.get_env("API_KEY")
```

This is evaluated when the release is built, on the build machine. The key there is usually missing, so production gets `nil`; or worse, the build machine's key gets baked into the artifact. Fix: read environment variables in `config/runtime.exs`, which runs on the target at boot.

5. Review checklist and sources

- ☐ Processes model state, concurrency or failure; pure logic stays in plain modules
- ☐ `Task.async` only where a task failure should crash the caller; otherwise `Task.Supervisor.async_nolink`
- ☐ No expensive functions passed into `Agent` calls
- ☐ Per-request and per-connection `GenServers` declare `restart: :temporary`
- ☐ No `String.to_atom/1` on external input
- ☐ Large data is processed with `Stream`, not loaded whole with `Enum`
- ☐ Environment-specific values are read in `config/runtime.exs`
- ☐ Every checklist from chapters 1-11 still applies

Sources

- GenServer: <https://hexdocs.pm/elixir/GenServer.html>
- Task: <https://hexdocs.pm/elixir/Task.html>
- Agent: <https://hexdocs.pm/elixir/Agent.html>
- DynamicSupervisor: <https://hexdocs.pm/elixir/DynamicSupervisor.html>
- Configuration and releases: <https://hexdocs.pm/elixir/config-and-releases.html>